Búsqueda DES con MPI

Felipe Aguilar · FelipeAP04 | Proyecto 2 · Tareas 6 y 7

Mediciones y método

Usamos el texto real de 352 palabras del grupo (2162 bytes, 2168 con relleno). La frase de búsqueda fue «computación paralela y distribuida». Probamos cinco llaves con 1, 2 y 4 procesos: 36 ejecuciones válidas, tres por combinación viable. Cada ejecución recuperó la llave esperada y el texto completo, verificado byte por byte.

Equipo: Apple M4 Pro, 14 núcleos y 24 GiB de RAM; macOS-26.6.2-arm64-arm-64bit. Open MPI 5.0.11, Apple clang 21.0.0 y optimización -O2. Todos los procesos se ejecutaron en una máquina, sin afinidad fija (--bind-to none). Inicio UTC: 2026-10-21T00:19:26.

El orden se aleatorizó con semilla 23195, con un calentamiento por cantidad de procesos. MPI_Wtime mide desde la barrera inicial hasta que el proceso 0 recibe la llave; excluye arranque MPI, lectura, descifrado final y finalización. No es el tiempo máximo de los procesos. Se registró aparte la duración externa de cada comando.

Promedios de búsqueda

Llave	np	Ganador	Media (s)	DE (s)	Speedup	Eficiencia
50 000	1	0	0.8070	0.1475	1.000	100.0 %
50 000	2	0	0.7317	0.0010	1.103	55.1 %
50 000	4	0	0.7346	0.0028	1.099	27.5 %
500 000	1	0	7.2364	0.0449	1.000	100.0 %
500 000	2	0	7.3185	0.0171	0.989	49.4 %
500 000	4	0	7.3472	0.0098	0.985	24.6 %
3 141 592	1	0	45.3373	0.0690	1.000	100.0 %
3 141 592	2	0	46.0129	0.0075	0.985	49.3 %
3 141 592	4	0	46.1428	0.1464	0.983	24.6 %
A	4	1	0.7360	0.0016	N/D	N/D
B	2	1	0.7318	0.0002	N/D	N/D
B	4	2	0.7337	0.0014	N/D	N/D

DE: desviación estándar muestral; tres repeticiones por fila. Speedup = media T_1 / media T_p; eficiencia = speedup / p. A = 2^{54} + 50000; B = 2^{55} + 50000. N/D: no se ejecutó la base de un proceso.

Se omitieron A con 1 y 2 procesos, y B con 1 proceso: su ganador necesitaría más de cuatro millones de intentos. Este filtro no modifica el espacio de 56 bits del programa. No se asignaron tiempos ni speedups a los casos omitidos.

Tiempos y speedup

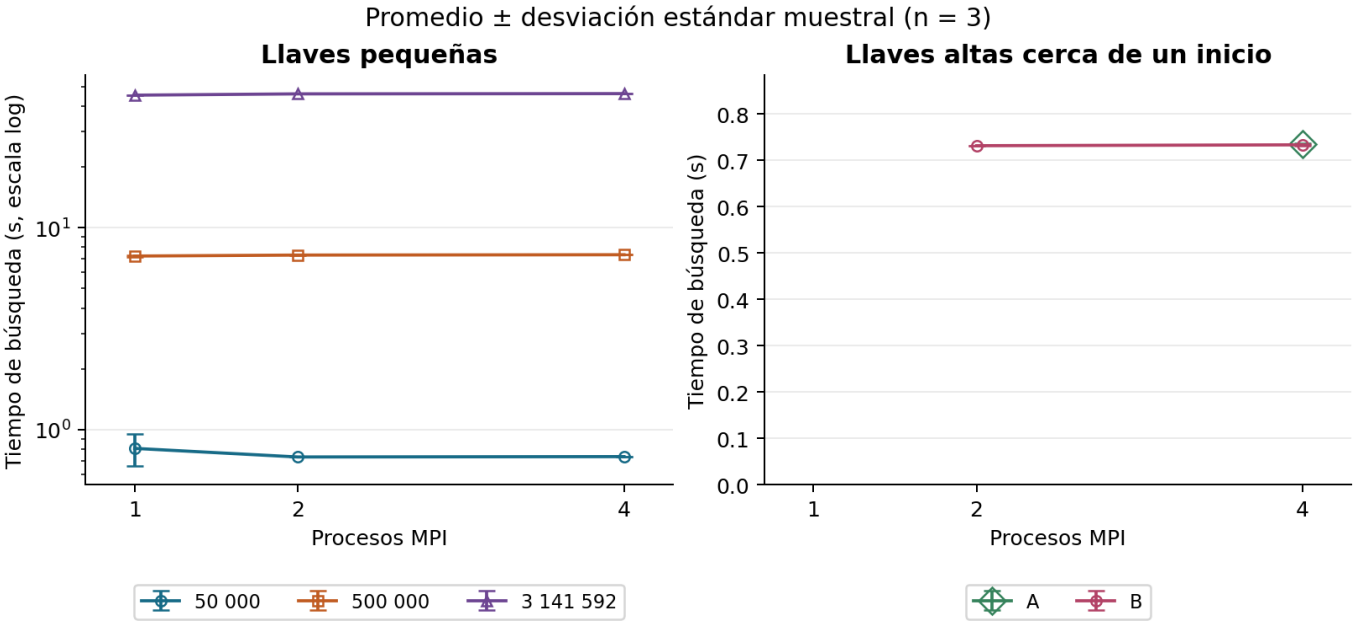


Figura 1. Promedio ± una desviación estándar muestral; no son intervalos de confianza. La izquierda usa escala logarítmica y la derecha lineal. A = 18014398509531984; B = 36028797019013968. Los puntos ausentes son casos omitidos, no tiempos cero.

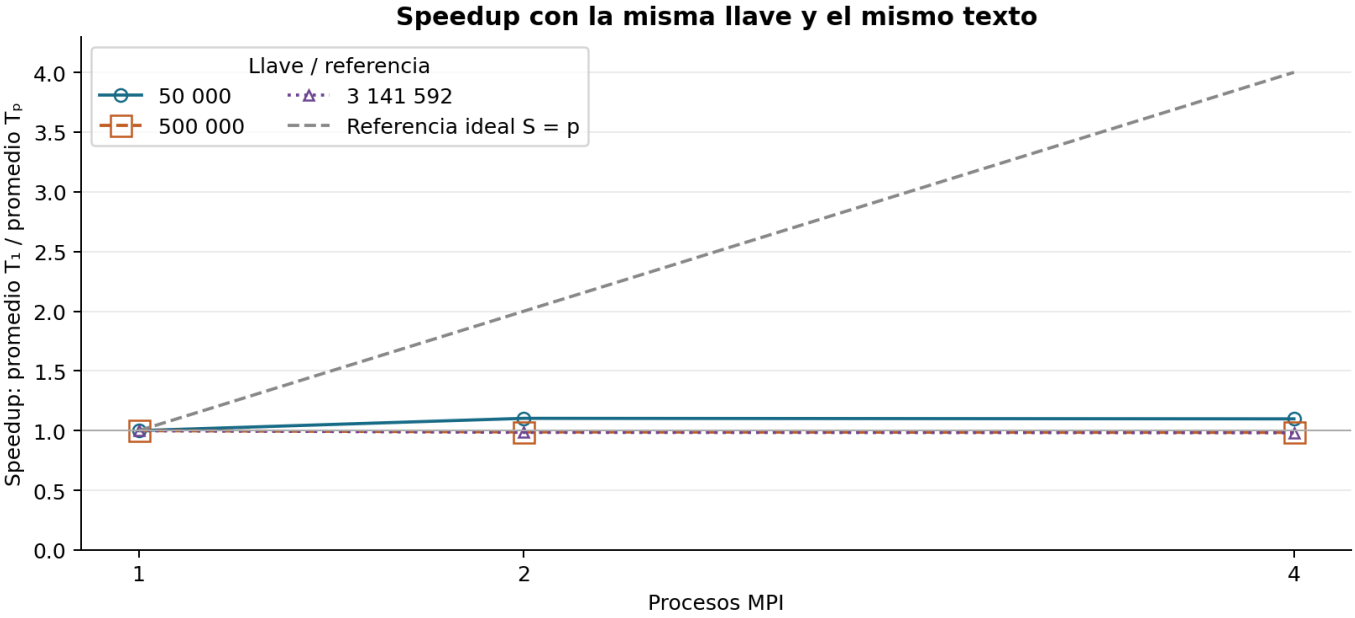


Figura 2. Cociente de promedios para una misma llave y el mismo texto. La referencia ideal S = p no representa una medición. Las llaves altas no aparecen porque no tienen una base medida con un proceso.

Impacto de la llave

La posición determina los intentos

El programa divide las 2^{56} llaves en bloques contiguos. El tamaño base es la división entera del espacio entre la cantidad de procesos; el último recibe el residuo. Cada proceso recorre su bloque en orden ascendente. Los intentos del ganador son la llave menos el inicio de su bloque, más uno. Por eso importa su posición dentro del bloque, no solamente el valor numérico de la llave.

Las llaves 50000, 500000 y 3141592 pertenecen al proceso 0 con 1, 2 y 4 procesos. Este prueba respectivamente 50001, 500001 y 3141593 candidatos en cualquiera de esas configuraciones. Los demás trabajan, pero no reducen ese recorrido. Con la llave original 3141592, el promedio fue 45.3373 s con un proceso y 46.1428 s con cuatro: speedup 0.983 y eficiencia 24.6 %. No observamos aceleración.

Una llave alta puede encontrarse antes

$A = 2^{54} + 50000$ cae en el bloque del proceso 1 con cuatro procesos: requiere 50001 intentos y promedia 0.7360 s. $B = 2^{55} + 50000$ requiere también 50001 intentos, pero la encuentra el proceso 1 con dos procesos y el proceso 2 con cuatro; sus promedios son 0.7318 y 0.7337 s. Son llaves mucho mayores que 3141592, pero están cerca del inicio de un bloque. Cerca del final habría que recorrer casi todo ese bloque. Cambiar la cantidad de procesos cambia estos límites.

No medimos A ni B con un proceso: exigirían más de 18 mil billones de intentos. Tampoco medimos A con dos procesos. Los tiempos paralelos sí son reales, pero no permiten reportar un speedup sin una base equivalente medida.

El speedup aparente puede engañar

Elegir llaves junto al inicio de un bloque favorece al recorrido paralelo frente al secuencial. Un speedup incluso superior al número de procesos podría deberse a ese cambio del orden de búsqueda, no a una mayor capacidad de cómputo. Nuestra campaña muestra esa dependencia; no demuestra aceleración superlineal.

La llave 50000 produjo un speedup de 1.099 con cuatro procesos, aunque el ganador hizo los mismos intentos. La base tuvo una muestra de 0.977363 s frente a otras de 0.721127 y 0.722594 s: su coeficiente de variación fue 18.28 %. Conservamos las tres muestras. No atribuimos esa aparente mejora a la distribución; no registramos información suficiente para explicar la corrida lenta.

Alcance y conclusión

Tres repeticiones describen esta campaña, no prueban significancia estadística. Las llaves se eligieron para contrastar posiciones, no como una muestra uniforme. No medimos red entre máquinas ni intentos totales: el contador corresponde solo al ganador. La planificación del sistema, la frecuencia de CPU y la comunicación también pueden afectar el tiempo; no aislamos sus aportes.

Antes de medir corregimos desplazamientos con signo, un límite de bloque y la finalización de recepciones. El cifrado original se conservó byte por byte; siete llaves se contrastaron con OpenSSL y el cifrado pasó el sanitizador UBSan. Concluimos que este reparto no acelera las llaves pequeñas evaluadas. Un reparto cíclico sería una alternativa para distribuir mejor sus candidatos, pero no se implementó ni se midió aquí.

Evidencia reproducible: bench/resultados/mediciones.csv, resumen.csv, entorno.json y logs/. El procedimiento está en bench/README.md; el código medido está en src/bruteforce_texto.c y src/des_util.h.